Why a Neural Pre-Decoder Fails for Surface-Code Decoding: Objective Misalignment, a Compute Roofline, and a Confidence Gate That Bounds the Damage

Nasir Ali[1*]

[1*]Embedded Systems Division, Centre for Development of Advanced Computing, Noida, 201307, India.

Corresponding author(s). E-mail(s): nasirali2607@gmail.com;

Abstract

Neural pre-decoders promise to cut the syndrome volume a matching decoder must resolve each surface-code round, easing a latency budget set by qubit coherence. We report a negative result with two identified causes. Training a convolutional pre-decoder on a GPU to 500,006,912 shots, over an order of magnitude beyond a prior CPU-only study, leaves it 3.79× worse than a PyMatching baseline: the loss converges, so undertraining is not the cause. Tracking both metrics across training shows why: they are correlated at -0.902, so the per-mechanism objective is measurably misaligned with logical error rate. Independently, a compute roofline on an AMD Alveo U55C bounds this architecture at 4.52 μs per shot using every DSP, so the microsecond target is unreachable by any implementation. We show a calibrated confidence gate bounds the damage, provably converging to the baseline, and quantify the headroom a perfect gate would exploit.

Keywords: Quantum error correction, surface code, neural decoder, FPGA, high-level synthesis, roofline analysis

1 Introduction

Surface-code quantum error correction requires a classical decoder to turn a stream of syndrome measurements into a correction within the time set by qubit coherence and

the syndrome-extraction period [9, 11]. Minimum-weight perfect matching (MWPM), implemented efficiently by PyMatching's sparse blossom algorithm [14, 15], is the standard choice. If decoding is slower than syndrome generation, undecoded rounds accumulate without bound, the decoder backlog problem [23, 24].

A neural *pre-decoder* is one proposed remedy: a small network resolves local detection events before an exact decoder handles the remainder [1, 6]. Recent work demonstrates real-time FPGA neural decoding on hardware [28], and GPU-resident pre-decoders reach microsecond-scale runtimes [6].

This paper reports that a faithful FPGA-oriented reimplementation of the pre-decoder concept does *not* work, and identifies why. Our contributions:

- **Scale is not the cause.** We train to 500,006,912 shots on a GPU, far beyond a prior CPU-only budget that named undertraining as its most likely failure cause. The pre-decoder remains 3.79× worse than baseline while training loss converges to 0.002197 (Section 7).
- **A compute roofline shows the latency target was never reachable.** The architecture costs 24,454,656 multiply-accumulates per shot; on the U55C's 9,024 DSPs at 300 MHz with INT8 packing, the floor is 4.52 μs per shot against a 1 μs budget (Section 5).
- **A confidence gate bounds the damage.** Escalating low-confidence shots to matching on the *raw* syndrome makes the hierarchical decoder provably no worse than the baseline, unlike an unconditional pre-decoder (Section 4).
- **An oracle bound separates gate quality from decoder value**, showing a perfect gate would reach 0.71× baseline but must find 95 useful shots among 1009 harmful ones.

2 Related work

Exact decoders.

MWPM decoding of the surface code descends from Dennis et al. [9] and the toric code [18], with the practical treatment of circuit-level noise and fault-tolerant thresholds developed by Fowler et al. [11]. PyMatching [14] made MWPM decoding widely usable, and its sparse blossom algorithm [15] brought single-core throughput to roughly a microsecond per round at distance 17. Union-find decoders trade a little accuracy for near-linear complexity [8]. We use sparse blossom both as our accuracy reference and as the fallback path, and re-measure it on our own hardware rather than quoting published figures.

Neural decoders.

Neural networks were applied to decoding early [25, 27] and have since reached state-of-the-art accuracy with recurrent transformer architectures [5], alongside experimental demonstrations of below-threshold operation [2]. The pre-decoder variant, in which a network removes easy detection events before an exact decoder resolves the rest, is the direct ancestor of this work [6]; Promatch [1] is the closest prior art to our confidence gate, adapting predecoding effort per shot. Our contribution is not the hierarchy, which is theirs, but the measurement of why an FPGA-oriented reimplementation of it fails.

Real-time and hardware decoding.

The backlog argument [24] motivates windowed and parallel decoding [23]; Battistel et al. survey the real-time problem [4]. Hardware decoders include lookup-table and ASIC-oriented designs [7, 19], cryogenic online decoders [20, 22], a scalable resource-efficient decoder deployed in a real system [3], and an FPGA neural decoder demonstrated in a superconducting control loop at 550 ns closed-loop latency [28]. Our roofline complements these by bounding what a given network can achieve on a given device before any of it is built.

Quantised inference.

Our precision assumptions follow standard quantised-inference practice [16, 17] and FPGA toolflows for small networks [10, 26]. Gate calibration uses temperature-scaling-style reliability analysis [13, 21].

3 Background

We simulate rotated surface-code memory circuits with Stim [12] (version 1.16.0) under circuit-level depolarising noise, and decode the resulting detector error model with PyMatching [15] (version 2.4.0). Monte Carlo collection uses `sinter` (version 1.16.0). Our target device is an AMD Alveo U55C (xcu55c-fsvh2892-2L-e), carrying 1,303,680 LUTs, 9,024 DSP slices, 2,016 block RAMs and 960 UltraRAMs across 3 super-logic regions, with 16 GB of HBM2 exposed as 32 pseudo-channels.

4 Design

The pre-decoder is a fully convolutional 3D network over the detector volume, with 113,358 parameters at width 64 and depth 2. It predicts which local error mechanisms fired; those corrections are applied to the syndrome, and the residual is decoded by MWPM.

4.1 The confidence gate

An unconditional pre-decoder has no recovery path: a wrong correction corrupts a syndrome the matcher would have decoded correctly. We therefore score each shot with seven cheap statistics (correction counts, decision margins, syndrome weight before and after) and a calibrated logistic model, and accept the pre-decoder path only when the score reaches a threshold τ. Escalated shots are decoded from the *raw* syndrome, not the corrected one. This yields two exact limits: at $\tau \to 0$ the system reduces to the unconditional pre-decoder, and at $\tau \to 1$ it reduces to the MWPM baseline. The hierarchical decoder is therefore bounded above by the baseline error rate for sufficiently large τ.

5 The compute roofline

The architecture requires 24,454,656 multiply-accumulates per shot. Counting only these, and charging nothing for data movement, activations, control or matching, the

U55C's 9,024 DSPs at 300 MHz give a floor of 4.52 μs per shot at INT8 with every DSP busy, sustaining 1.33 logical qubits. At a realistic 50 % DSP utilisation these become 9.03 μs and 0.664 qubits. The 1 μs round budget is therefore unreachable for this architecture regardless of implementation quality. Inverting the bound gives a design target: width 52 sustains one logical qubit, width 25 sustains four, and width 12 sustains sixteen (Fig. 5).

6 Methodology

All experiments run on an AMD EPYC 7742 64-Core Processor with 64 logical cores and an NVIDIA RTX A4000. FPGA tooling is Vitis 2023.2 with XRT 2.15.225. Results carry a provenance tier: **T1** measured on hardware, **T2** post-place-and-route, **T3** HLS estimate, **T4** simulation or analytical model. Logical error rates are Monte Carlo estimates from simulated circuits and are therefore T4; device resource counts and idle power are T1.

Baselines span 44 cells at distances 3, 5, 7, 9, totalling 164,558,040 shots and 47,591 observed logical errors, with Clopper–Pearson 95 % intervals.

7 Results

7.1 Baseline and threshold

The MWPM baseline behaves correctly: below threshold the logical error rate falls with distance and above it rises (Fig. 1). A finite-size-scaling collapse (Fig. 2) gives $p_{\text{th}} = 0.699\,\%$ with bootstrap 95 % interval $[0.688, 0.71]\,\%$ and $\nu = 1.16$ over 500 converged resamples. Varying the fitting window moves the estimate over $[0.65, 0.699]\,\%$, a systematic wider than the statistical interval, so we quote both.

7.2 Training at scale does not help

At 500,006,912 shots (70.3 minutes at 118,524 shots/s), the training loss converges to 0.002197. Evaluated on 100,000 held-out shots at $d = 5$, $p = 0.003$, the baseline reaches 0.00328 while the unconditional pre-decoder reaches 0.01242, a factor of 3.79 worse. Because the loss has converged, this is not undertraining: the per-mechanism objective is misaligned with logical error rate, and optimising it harder makes the network apply more confident wrong corrections.

7.3 Training actively degrades decoding

Ruling out budget and capacity leaves the objective, but that is an inference until the two quantities are watched together. Misalignment is a claim about how two curves move relative to one another, so we evaluated 20 checkpoints from a single run against one fixed held-out set of 200,000 shots, pairing the training loss with the resulting logical error rate (Fig. 3).

The two disagree, strongly. Over the run the loss falls monotonically from 0.0218 to 0.00255 while the pre-decoder's error rate rises from 1.33× baseline to 3.56×, peaking

at 3.92×. Their correlation is -0.902. The best decoder in the entire run is the least-trained checkpoint, at 10,027,008 shots.

The mechanism is visible in what the corrections do. At the first checkpoint the pre-decoder rescues 62 shots the matcher gets wrong and corrupts 297 it would have got right; by the last it rescues 170 and corrupts 2000. Training teaches the network to intervene more often, and the additional interventions are overwhelmingly harmful. Minimising per-mechanism cross-entropy is not the same as minimising logical error rate, and past a very early point the two objectives pull in opposite directions.

One nuance argues against reading this as the network simply getting worse. The oracle bound *improves* across the same run, from 0.913× baseline to 0.762×: training does add information that a perfect gate could exploit. What degrades is the network's own decision rule about when to use it.

7.4 Capacity does not help either

Training budget is one axis; model capacity is another. We trained 3 further networks spanning widths 12 to 52, covering more than an order of magnitude in multiply-accumulates, and evaluated each identically. The logical error rate is flat: the narrowest model reaches 3.6× baseline and the widest 3.87×, a spread of only 0.27 (Fig. 4). The narrowest network is in fact marginally the best of the four.

This matters twice over. Scientifically, it is a second independent axis on which the failure does not move, which together with the training-budget result rules out both undertraining and insufficient capacity and leaves the objective as the explanation. Practically, it inverts the deployment argument: since accuracy is indifferent to width while sustainable qubits per card is not (Section 5), the cheapest network dominates. Width 12 is precisely the width the roofline identifies as sustaining sixteen logical qubits per card, against under one at the width the model was originally sized at, for no measurable accuracy cost.

7.5 The gate bounds the damage, and the oracle bounds the gate

Sweeping τ moves the decoder monotonically from the unconditional result to the baseline, reaching full escalation at $\tau = 0.999$. The gate is well calibrated, with expected calibration error 0.00226. An oracle that chose the better path per shot would reach 0.00233 (0.71× baseline), so the pre-decoder does carry complementary information; but it rescues only 95 shots while corrupting 1009, so exploiting that headroom demands a discrimination the present feature set cannot provide.

7.6 A preliminary synthesis of the deployable width

The roofline says width 12 is the configuration worth building, so we parameterised the HLS kernel by channel width and put that configuration through Vitis C-synthesis. Against a 3.33 ns target the tool estimates 2.76 ns and a worst-case latency of 456 μs, using 187,131 LUTs (14 %), 159,474 flip-flops, 113 DSPs (1.3 %) and 60 block RAMs.

That is 1,306× the 0.349 μs bound for the same architecture, and the reason is visible in the DSP column: the design instantiates 113 DSPs against the 40× larger

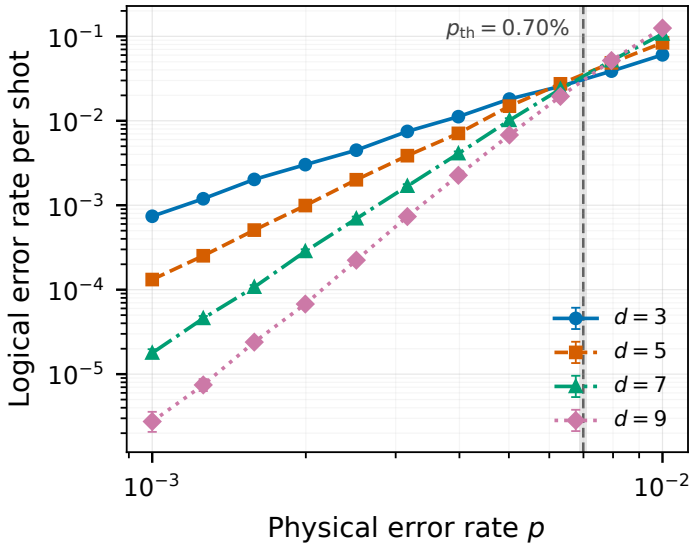

Fig. 1 Logical error rate versus physical error rate for the MWPM baseline at distances 3, 5, 7, 9, with Clopper–Pearson 95 % intervals. The dashed line marks the fitted threshold. Provenance: T4

budget the bound assumes. The kernel deliberately serialises its input-channel reduction to avoid a block-RAM blow-up, and the cost of that choice is most of the gap. The two obstacles this paper identifies are therefore independent and compound: an architectural bound that no implementation can beat, and an implementation that is far from even that bound.

This result is preliminary and is not evidence of a working decoder. At the time of writing the kernel's C-simulation disagrees with the golden model for this configuration, so while the loop structure that determines resources and latency is settled, we cannot claim the synthesised datapath computes correct corrections, and the figures above may move once that is resolved. They are reported as T3 estimates with functional verification outstanding rather than withheld, because the size of the implementation gap is itself the useful quantity and is unlikely to be explained by a correctness bug of this kind.

7.7 Scope and limitations of the hardware evaluation

The roofline is analytical (T4), built from T1 device counts. We report no new bitstream here; the prior implementation this work builds on measured 13–15.3 ms per shot using 331 DSPs at 100 MHz, leaving large implementation headroom that the roofline shows would still be insufficient. Logical error rates come from simulated circuits, not a quantum device.

8 Discussion

Two independent obstacles emerge. The accuracy failure is an objective problem: the network is trained on a proxy that does not track logical error rate. The latency failure

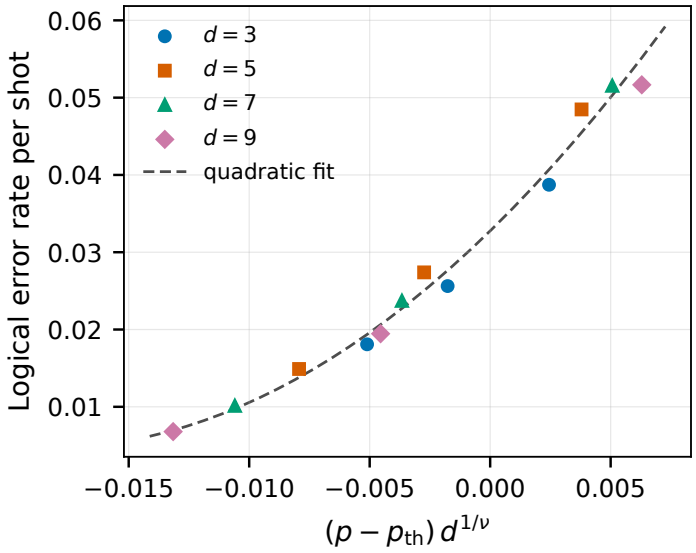

Fig. 2 Finite-size-scaling collapse of the baseline data onto a single quadratic. Provenance: T4

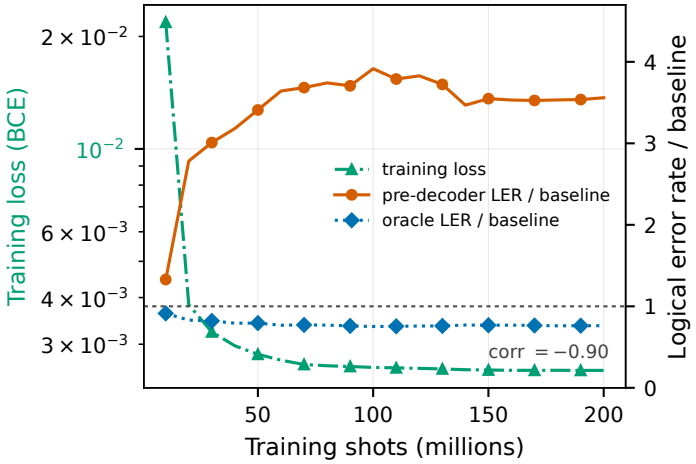

Fig. 3 Training loss (left axis) against logical error rate relative to baseline (right axis) over a single training run. The objective being minimised improves while the quantity the decoder exists to minimise gets worse; the oracle bound improves, showing the information is present but increasingly misused. Provenance: T4

is an architectural one: the network is too large for the device by roughly an order of magnitude. Neither is addressed by more compute, and the second is not addressed by better HLS. A loss penalising logical errors directly, and a network narrow enough to satisfy the roofline, are the natural next steps.

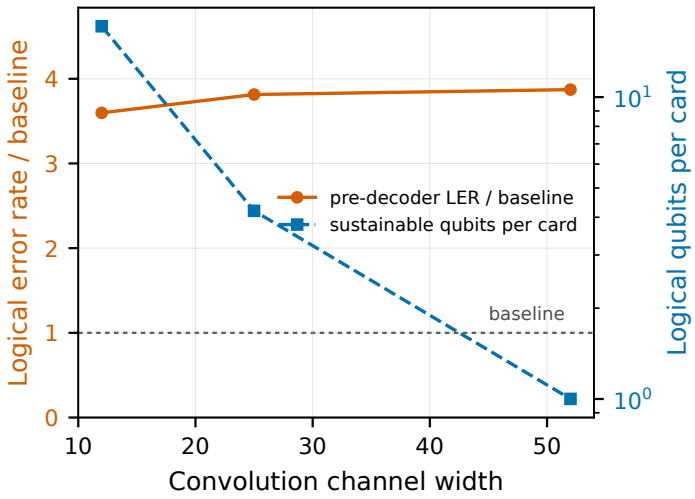

Fig. 4 Logical error rate relative to baseline (left axis) and sustainable logical qubits per card (right axis) against convolution width. Accuracy is flat while deployability varies by more than an order of magnitude. Provenance: T4

9 Conclusion

We report, and diagnose, a negative result for neural pre-decoding of the surface code on an HBM-class FPGA. Training at scale does not rescue accuracy; the architecture cannot meet the round budget on this device at any implementation quality; and a calibrated confidence gate, while unable to make the pre-decoder useful, does make it safe.

Declarations

Funding: This work received no external funding.

Competing interests: The author declares no competing interests.

Data availability: All result files, figures and generated numbers are available in the public repository accompanying this paper.

Code availability: All code is available in the same repository.

Author contributions: N.A. conceived the study, implemented the software, performed the experiments and wrote the manuscript.

Ethics approval: Not applicable.

References

[1] Narges Alavisamani, Suhas Vittal, Ramin Ayanzadeh, Poulami Das, and Moinuddin Qureshi. Promatch: Extending the reach of real-time quantum error correction with adaptive predecoding, 2024.

[2] Rajeev and Acharya, Dmitry A. Abanin, Laleh Aghababaie-Beni, Igor Aleiner, Trond I. Andersen, Markus Ansmann, Frank Arute, Kunal Arya, Abraham Asfaw,

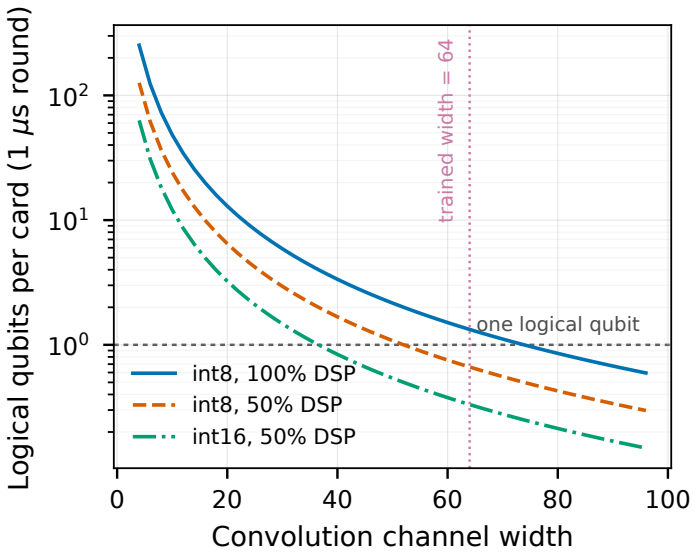

Fig. 5 Sustainable logical qubits per card versus convolution width, from the compute roofline. Provenance: T4 bound from T1 device counts

Nikita Astrakhantsev, Juan Atalaya, Ryan Babbush, Dave Bacon, Brian Ballard, Joseph C. Bardin, Johannes Bausch, Andreas Bengtsson, Alexander Bilmes, Sam Blackwell, Sergio Boixo, Gina Bortoli, Alexandre Bourassa, Jenna Bovaird, Leon Brill, Michael Broughton, David A. Browne, Brett Buchea, Bob B. Buckley, David A. Buell, Tim Burger, Brian Burkett, Nicholas Bushnell, Anthony Cabrera, Juan Campero, Hung-Shen Chang, Yu Chen, Zijun Chen, Ben Chiaro, Desmond Chik, Charina Chou, Jahan Claes, Agnetta Y. Cleland, Josh Cogan, Roberto Collins, Paul Conner, William Courtney, Alexander L. Crook, Ben Curtin, Sayan Das, Alex Davies, Laura De Lorenzo, Dripto M. Debroy, Sean Demura, Michel Devoret, Agustin Di Paolo, Paul Donohoe, Ilya Drozdov, Andrew Dunsworth, Clint Earle, Thomas Edlich, Alec Eickbusch, Aviv Moshe Elbag, Mahmoud Elzouka, Catherine Erickson, Lara Faoro, Edward Farhi, Vinicius S. Ferreira, Leslie Flores Burgos, Ebrahim Forati, Austin G. Fowler, Brooks Foxen, Suhas Ganjam, Gonzalo Garcia, Robert Gasca, Élie Genois, William Giang, Craig Gidney, Dar Gilboa, Raja Gosula, Alejandro Grajales Dau, Dietrich Graumann, Alex Greene, Jonathan A. Gross, Steve Habegger, John Hall, Michael C. Hamilton, Monica Hansen, Matthew P. Harrigan, Sean D. Harrington, Francisco J. H. Heras, Stephen Heslin, Paula Heu, Oscar Higgott, Gordon Hill, Jeremy Hilton, George Holland, Sabrina Hong, Hsin-Yuan Huang, Ashley Huff, William J. Huggins, Lev B. Ioffe, Sergei V. Isakov, Justin Iveland, Evan Jeffrey, Zhang Jiang, Cody Jones, Stephen Jordan, Chaitali Joshi, Pavol Juhas, Dvir Kafri, Hui Kang, Amir H. Karamlou, Kostyantyn Kechedzhi, Julian Kelly, Trupti Khaire, Tanuj Khattar, Mostafa Khezri, Seon Kim, Paul V. Klimov, Andrey R. Klots, Bryce Kobrin, Pushmeet Kohli, Alexander N. Korotkov, Fedor Kostritsa, Robin Kothari, Borislav Kozlovskii, John Mark Kreikebaum, Vladislav D. Kurilovich,

Nathan Lacroix, David Landhuis, Tiano Lange-Dei, Brandon W. Langley, Pavel Laptev, Kim-Ming Lau, Loïck Le Guevel, Justin Ledford, Joonho Lee, Kenny Lee, Yuri D. Lensky, Shannon Leon, Brian J. Lester, Wing Yan Li, Yin Li, Alexander T. Lill, Wayne Liu, William P. Livingston, Aditya Locharla, Erik Lucero, Daniel Lundahl, Aaron Lunt, Sid Madhuk, Fionn D. Malone, Ashley Maloney, Salvatore Mandrà, James Manyika, Leigh S. Martin, Orion Martin, Steven Martin, Cameron Maxfield, Jarrod R. McClean, Matt McEwen, Seneca Meeks, Anthony Megrant, Xiao Mi, Kevin C. Miao, Amanda Mieszala, Reza Molavi, Sebastian Molina, Shirin Montazeri, Alexis Morvan, Ramis Movassagh, Wojciech Mruczkiewicz, Ofer Naaman, Matthew Neeley, Charles Neill, Ani Nersisyan, Hartmut Neven, Michael Newman, Jiun How Ng, Anthony Nguyen, Murray Nguyen, Chia-Hung Ni, Murphy Yuezhen Niu, Thomas E. O'Brien, William D. Oliver, Alex Opremcak, Kristoffer Ottosson, Andre Petukhov, Alex Pizzuto, John Platt, Rebecca Potter, Orion Pritchard, Leonid P. Pryadko, Chris Quintana, Ganesh Ramachandran, Matthew J. Reagor, John Redding, David M. Rhodes, Gabrielle Roberts, Eliott Rosenberg, Emma Rosenfeld, Pedram Roushan, Nicholas C. Rubin, Negar Saei, Daniel Sank, Kannan Sankaragomathi, Kevin J. Satzinger, Henry F. Schurkus, Christopher Schuster, Andrew W. Senior, Michael J. Shearn, Aaron Shorter, Noah Shutty, Vladimir Shvarts, Shraddha Singh, Volodymyr Sivak, Jindra Skruzny, Spencer Small, Vadim Smelyanskiy, W. Clarke Smith, Rolando D. Somma, Sofia Springer, George Sterling, Doug Strain, Jordan Suchard, Aaron Szasz, Alex Sztein, Douglas Thor, Alfredo Torres, M. Mert Torunbalci, Abeer Vaishnav, Justin Vargas, Sergey Vdovichev, Guifre Vidal, Benjamin Villalonga, Catherine Vollgraff Heidweiller, Steven Waltman, Shannon X. Wang, Brayden Ware, Kate Weber, Travis Weidel, Theodore White, Kristi Wong, Bryan W. K. Woo, Cheng Xing, Z. Jamie Yao, Ping Yeh, Bicheng Ying, Juhwan Yoo, Noureldin Yosri, Grayson Young, Adam Zalcman, Yaxing Zhang, Ningfeng Zhu, and Nicholas Zobrist. Quantum error correction below the surface code threshold. *Nature*, 638(8052):920–926, 2024.

[3] Ben Barber, Kenton M. Barnes, Tomasz Bialas, Okan Buğdaycı, Earl T. Campbell, Neil I. Gillespie, Kauser Johar, Ram Rajan, Adam W. Richardson, Luka Skoric, Canberk Topal, Mark L. Turner, and Abbas B. Ziad. A real-time, scalable, fast and resource-efficient decoder for a quantum computer. *Nature Electronics*, 8(1):84–91, 2025.

[4] F Battistel, C Chamberland, K Johar, R W J Overwater, F Sebastiano, L Skoric, Y Ueno, and M Usman. Real-time decoding for fault-tolerant quantum computing: progress, challenges and outlook. *Nano Futures*, 7(3):032003, 2023.

[5] Johannes Bausch, Andrew W. Senior, Francisco J. H. Heras, Thomas Edlich, Alex Davies, Michael Newman, Cody Jones, Kevin Satzinger, Murphy Yuezhen Niu, Sam Blackwell, George Holland, Dvir Kafri, Juan Atalaya, Craig Gidney, Demis Hassabis, Sergio Boixo, Hartmut Neven, and Pushmeet Kohli. Learning high-accuracy error decoding for quantum processors. *Nature*, 635(8040):834–840, 2024.

[6] Christopher Chamberland, Jan Olle, Muyuan Li, Scott Thornton, and Igor Baratta. Fast and accurate ai-based pre-decoders for surface codes, 2026.

[7] Poulami Das, Aditya Locharla, and Cody Jones. Lilliput: A lightweight low-latency lookup-table based decoder for near-term quantum error correction, 2021.

[8] Nicolas Delfosse and Naomi H. Nickerson. Almost-linear time decoding algorithm for topological codes. *Quantum*, 5:595, 2021.

[9] Eric Dennis, Alexei Kitaev, Andrew Landahl, and John Preskill. Topological quantum memory. *Journal of Mathematical Physics*, 43(9):4452–4505, 2002.

[10] J. Duarte, S. Han, P. Harris, S. Jindariani, E. Kreinar, B. Kreis, J. Ngadiuba, M. Pierini, R. Rivera, N. Tran, and Z. Wu. Fast inference of deep neural networks in fpgas for particle physics. *Journal of Instrumentation*, 13(07):P07027–P07027, 2018.

[11] Austin G. Fowler, Matteo Mariantoni, John M. Martinis, and Andrew N. Cleland. Surface codes: Towards practical large-scale quantum computation. *Physical Review A*, 86(3), 2012.

[12] Craig Gidney. Stim: a fast stabilizer circuit simulator. *Quantum*, 5:497, 2021.

[13] Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. On calibration of modern neural networks, 2017.

[14] Oscar Higgott. Pymatching: A python package for decoding quantum codes with minimum-weight perfect matching, 2021.

[15] Oscar Higgott and Craig Gidney. Sparse blossom: correcting a million errors per core second with minimum-weight matching. *Quantum*, 9:1600, 2025.

[16] Itay Hubara, Matthieu Courbariaux, Daniel Soudry, Ran El-Yaniv, and Yoshua Bengio. Quantized neural networks: Training neural networks with low precision weights and activations, 2016.

[17] Benoit Jacob, Skirmantas Kligys, Bo Chen, Menglong Zhu, Matthew Tang, Andrew Howard, Hartwig Adam, and Dmitry Kalenichenko. Quantization and training of neural networks for efficient integer-arithmetic-only inference, 2017.

[18] A.Yu. Kitaev. Fault-tolerant quantum computation by anyons. *Annals of Physics*, 303(1):2–30, 2003.

[19] Andrzej Krawiecki and Tomasz Gradowski. The q-neighbor ising model on multiplex networks with partial overlap of nodes, 2023.

[20] Matteo Lostaglio and Alessandro Ciani. Error mitigation and quantum-assisted simulation in the error corrected regime, 2021.

[21] Alexandru Niculescu-Mizil and Rich Caruana. Predicting good probabilities with supervised learning. In *Proceedings of the 22nd international conference on Machine learning - ICML '05*, ICML '05, page 625–632. ACM Press, 2005.

[22] Jethin J. Pulikkottil, Arul Lakshminarayan, Shashi C. L. Srivastava, Maximilian F. I. Kieler, Arnd Bäcker, and Steven Tomsovic. Quantum coherence controls the nature of equilibration in coupled chaotic systems, 2022.

[23] Luka Skoric, Dan E. Browne, Kenton M. Barnes, Neil I. Gillespie, and Earl T. Campbell. Parallel window decoding enables scalable fault tolerant quantum computation. *Nature Communications*, 14(1), 2023.

[24] Barbara M. Terhal. Quantum error correction for quantum memories. *Reviews of Modern Physics*, 87(2):307–346, 2015.

[25] Giacomo Torlai and Roger G. Melko. Neural decoder for topological codes. *Physical Review Letters*, 119(3), 2017.

[26] Yaman Umuroglu, Nicholas J. Fraser, Giulio Gambardella, Michaela Blott, Philip Leong, Magnus Jahre, and Kees Vissers. Finn: A framework for fast, scalable binarized neural network inference, 2016.

[27] Savvas Varsamopoulos, Ben Criger, and Koen Bertels. Decoding small surface codes with feedforward neural networks. *Quantum Science and Technology*, 3(1):015004, 2017.

[28] Xiaohan Yang, Xuandong Sun, Zhiyi Wu, Jiawei Zhang, Ji Jiang, Xiayu Linpeng, Yuxuan Zhou, Ji Chu, Jingjing Niu, Youpeng Zhong, Song Liu, and Dapeng Yu. Real-time surface-code error correction using an fpga-based neural-network decoder, 2026.